Software Transactional Memory (STM)

Diogo Gomes de Araújo[1]

[1]Faculdade de Ciências da Universidade do Porto

May 29, 2026

U.PORTO

FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

“The free lunch is over”

Locks

Locks or *mutexes* give a necessary but **"pessimistic"** boudary for **data integrity** where:

- **Critical sections** where shared data is accessed are isolation through a **lock-based system**;
- It is the programmer's task to identify and anotate such sections;

Though simple, locks come with limitations:

- Overreliance on **mutual exclusion** imposes a rigid **sequential bottleneck**;
- Concurrent programs relying on locks are **not trivially composable** into larger ones;
- Naive implementations may result in **resource starvation**, namely *deadlocks*, **priority inversion** and **kernel-level thread suspension**;

Locks on a stock exchange's order book

Recent *Nasdaq* market summaries show the extraordinary scale at which modern exchanges operate, processing around 70 million orders daily.

Table: *Nasdaq* Daily Market Summary on May 19, 2026

Metric	Share Volume	Dollar Volume
Total Volume	10,073,462,044	$527,866,082,028
Block Volume	2,399,217,446	–

Additional Statistics	Value
Number of Issues	5,622
Number of MPs	605
Total Trades	69,442,435
Block Trades	77,017

,

Locks on a stock exchange's order book

An unwise approach for modeling concurrent access in an imperative programming language, such as *Rust*, using locks as the primary synchronization primitive would be:

```
struct OrderBook<Order> {
  bid_orders: Arc<Mutex<PriorityQueue<Order>>>,
  ask_orders: Arc<Mutex<PriorityQueue<Order>>>,
}
```

May cause a deadlock if concurrent access is not well though out:

```
if let Ok(mut bid_orders) = order_book.bid_orders.lock() {
    bid_orders.insert(order);

    let lowest_ask = order_book.ask_orders.lock()?.remove();
    ...
}
```

“A transaction is a finite sequence of local and shared memory machine instructions”

Hardware Transactional Memory

In Transactional Memory (TM):

- Database operations are handled through sequences of `read_transactional` `write_transactional` instructions that compose a transaction;
- Each transaction either **terminates successfully**, when all operations occur, updating the **shared memory state**, or **aborts** and is re-queued for later execution when one or more fail to run;
- A **transactional cache** stores writes temporarily until a `commit` or `abort` instruction was called.

Though simple, this model allows for implementing **non-blocking** and **lock-free** concurrent access to shared data structures.

Hardware Transactional Memory

Comes with its own set of limitations:

- Requires **hardware-specific synchronization mechanisms** to create lock-free shared memory locations on multiprocessor systems;
- If a transaction's dataset **exceeded the size of the cache**, it unavoidably leads to **abortion**.

Software Transactional Memory (STM)

STM expands upon this framework by providing a software-based abstraction for managing concurrent access to shared memory without relying on hardware support.

In STM:

- A thread executes a transaction **in isolation**, on a **local copy** of the shared memory
- If no other thread has written to the same memory addresses during execution, the changes are **committed**, otherwise, the transaction is **aborted**;
- The reader is tasked with re-evaluating data consistency after executing an atomic instruction sequence;

By treating memory operations as **atomic transactions** rather than protected critical sections, STM **mitigates the risks** of **priority inversion** and **deadlock**.

How STM relates with Functional Programming

Since transactions encapsulate effects within an ephemeral log, they behave as first-class values that can be sequenced, combined, and passed as arguments before any effect is committed to shared state.

Concurrency in Haskell

Concurrency primitives are encoded in `Control.Concurrent` using monadic IO properties to enforce strict manipulation of side effects, ensuring that operations such as reading shared state or spawning threads remain predictable:

```
forkIO :: IO a -> IO ThreadId
```

STM Monad

Transactions, in Haskell, are composed using the STM monad, provided by `Control.Concurrent.STM`. A transactional variable of type `TVar` can be atomically mutated and read through sequences of `readTVar` and `writeTVar` operations:

```
readTVar  :: TVar a -> STM a
writeTVar :: TVar a -> a -> STM ()
```

As such, transactions are composed into pure `STM` values and only executed when lifted into `IO`, using the `atomically` operator defined by the following type:

```
atomically :: STM a -> IO a
```

This mechanism ensures that all transactional actions are **executed atomically**, meaning that **intermediate states are never observable** by other threads.

Modeling a transactional order book

The design of a stock exchange order book is a natural application of STM, as it involves frequent concurrent modifications to shared state under strict consistency requirements.

A transacitonal order book could be implemented as:

```
data TOrderBook c = TOrderBook { bidOrders :: TPriorityQueue (Order c),
                                 askOrders :: TPriorityQueue (Order c),
                                 clock     :: TVar (Timestamp) }
```

Unlike a lock-based design, **no explicit synchronisation primitives** guard this state.

Instead, the **runtime detects conflicts dynamically** and **retries computations** as needed, without programmer intervention.

Modeling a transactional order book

Each order pairs a semantic header with a timestamp. The header encodes whether the order is a buy or sell request and its associated limit price:

```
data OrderType = Buy | Sell
    deriving (Show, Eq)

data Header c = Header { limitAmount :: c,
                         orderType   :: OrderType }

data Order c = Order { header    :: Header c,
                       timestamp :: Timestamp }
```

Modeling a transactional order book

We derive an `Ord` instance to ensure that better-priced orders are prioritized, with ties resolved in FIFO order through timestamps:

```
instance Ord c => Ord (Order c) where
    compare ordA ordB =
    compare (limitAmount $ header ordA, timestamp ordA)
        (limitAmount $ header ordB, timestamp ordB)
```

To preserve **global ordering consistency**, we assign each order a **timestamp atomically** inside the `STM` context:

```
stamp :: Ord c => TOrderBook c -> Header c -> STM (Order c)
stamp ordBook h = do
    ts    <- readTVar $ clock ordBook
    _     <- writeTVar (clock ordBook) (ts + 1)
    return $ newOrder h ts
```

Modeling a transactional order book

Initialization creates empty transactional priority queues alongside a fresh clock, composing naturally within the `STM` monad:

```
instance Ord c => Ord (Order c) where
    compare ordA ordB =
    compare (limitAmount $ header ordA, timestamp ordA)
        (limitAmount $ header ordB, timestamp ordB)
```

Modeling a transactional order book

Timestamp assignment and **queue insertion** are merged into a **single transaction**, ensuring intermediate states are **never visible** to other threads:

```
addTOrderBook :: Ord c => Header c -> TOrderBook c -> STM ()
addTOrderBook hd ordBook = do
    ord <- stamp ordBook hd
    case orderType hd of
        Buy  -> insertTPriorityQueue ord (bidOrders ordBook)
        Sell -> insertTPriorityQueue ord (askOrders ordBook)

removeTOrderBook :: Ord c => OrderType -> TOrderBook c -> STM (Maybe (Order
c))
removeTOrderBook t ordBook = case t of
    Buy  -> pollTPriorityQueue $ bidOrders ordBook
    Sell -> pollTPriorityQueue $ askOrders ordBook
```